

神经网络与 SVM

2312473 李一珩

2026-04-29

数据准备与加载

根据课本（*Machine Learning with R*）第七章的设定：

1. `concrete.csv` 是神经网络的回归案例，目标是预测混凝土的抗压强度 (`strength`)。
2. `letterdata.csv` 是 SVM 的分类案例，目标是识别字母。

我们首先加载这两个数据集。

```
# 读取数据（这一步可以使用张老师上课说的 Session-set working directory-to source location 实现，避免每
concrete <- read.csv("D:/zlr/Machine-Learning-with-R-Fourth-Edition-main/Chapter 07/concrete.csv")
letters <- read.csv("D:/zlr/Machine-Learning-with-R-Fourth-Edition-main/Chapter 07/letterdata.csv")

# 神经网络对数据尺度非常敏感，因此需要先将 concrete 数据归一化到 [0, 1] 区间
normalize <- function(x) {
  return ((x - min(x)) / (max(x) - min(x)))
}
concrete_norm <- as.data.frame(lapply(concrete, normalize))

# 划分训练集 (75%) 和测试集 (25%)
set.seed(12345)
train_idx <- sample(1:nrow(concrete_norm), 0.75 * nrow(concrete_norm))
concrete_train <- concrete_norm[train_idx, ]
concrete_test <- concrete_norm[-train_idx, ]
```

问题 1：对课本的神经网络例子中的数据，试着构造一个线性回归模型。进一步比较线性回归模型和神经网络的预测结果的差异。

对于混凝土强度的预测问题，我们分别训练一个多元线性回归模型和一个单隐藏层的神经网络模型。

1.1 训练模型

```
# 1. 构造线性回归模型
lm_model <- lm(strength ~ ., data = concrete_train)

# 2. 构造神经网络模型 (包含 1 个隐藏节点)
# 这里选择 1 个节点是为了方便后续在问题 2 中进行手动计算验证
nn_model <- neuralnet(strength ~ cement + slag + ash + water + superplastic +
                      coarseagg + fineagg + age,
                      data = concrete_train, hidden = 1)
```

1.2 预测结果对比

```
# 线性模型预测
lm_pred <- predict(lm_model, concrete_test)

# 神经网络预测
nn_pred_obj <- compute(nn_model, concrete_test[1:8])
nn_pred <- nn_pred_obj$net.result

# 计算真实值与预测值的相关系数 (Correlation) 和 均方误差 (MSE) 作为评估指标
actual_strength <- concrete_test$strength

cat("---- 线性回归模型性能 ---\n")
```

```
## ---- 线性回归模型性能 ----
```

```
cat(" 相关系数 (Cor):", cor(lm_pred, actual_strength), "\n")
```

```
## 相关系数 (Cor): 0.7624508
```

```
cat(" 均方误差 (MSE):", mean((lm_pred - actual_strength)^2), "\n\n")
```

```
## 均方误差 (MSE): 0.01877737
```

```
cat("---- 神经网络模型性能 ----\n")
```

```
## --- 神经网络模型性能 ---
```

```
cat(" 相关系数 (Cor):", cor(nn_pred, actual_strength), "\n")
```

```
## 相关系数 (Cor): 0.7969299
```

```
cat(" 均方误差 (MSE):", mean((nn_pred - actual_strength)^2), "\n")
```

```
## 均方误差 (MSE): 0.01643335
```

结论与差异比较: 从测试集的评估结果可以看出, 即使是只有一个隐藏节点的简单神经网络, 其预测能力(相关系数更高, MSE 更低)通常也优于传统的多元线性回归。这是因为混凝土的配比与最终强度之间往往存在复杂的非线性化学及物理关系, 线性回归只能拟合超平面, 而神经网络可以通过激活函数捕捉这些非线性特征。如果增加隐藏层节点数, 神经网络的优势会进一步扩大。

问题 2: 在课本的神经网络简单例子中, 验证一下预测值是如何得到的? 这涉及到如何找出神经网络的参数。

为了验证预测值的计算逻辑, 我们将提取刚才训练好的 `nn_model` 的权重 (Weights) 和偏置 (Biases), 并提取测试集的第一条数据, 手动进行前向传播 (Forward Propagation) 计算。

```
# 1. 提取测试集的第一个样本 (不包含目标变量 strength)
```

```
x_input <- as.numeric(concrete_test[1, 1:8])
```

```
# 2. 提取神经网络的权重和偏置
```

```
# nn_model$weights[[1]][[1]] 包含输入层到隐藏层的截距 (偏置) 和权重
```

```
bias_hidden <- nn_model$weights[[1]][[1]][1, 1] # 隐藏层的偏置项
```

```
weights_in_to_hid <- nn_model$weights[[1]][[1]][-1, 1] # 8 个输入特征的权重
```

```
# nn_model$weights[[1]][[2]] 包含隐藏层到输出层的截距 (偏置) 和权重
```

```

bias_output <- nn_model$weights[[1]][[2]][1, 1]      # 输出层的偏置项
weight_hid_to_out <- nn_model$weights[[1]][[2]][-1, 1] # 隐藏层到输出的权重

# 3. 手动执行前向传播 (Forward Propagation)
# 激活函数: neuralnet 默认使用 sigmoid
sigmoid <- function(x) { 1 / (1 + exp(-x)) }

# 第 1 步: 计算隐藏层节点的输入及激活后的输出
hidden_input <- sum(x_input * weights_in_to_hid) + bias_hidden
hidden_output <- sigmoid(hidden_input)

# 第 2 步: 计算最终输出层的预测值 (对于回归问题, 输出层通常不加激活函数)
final_prediction <- hidden_output * weight_hid_to_out + bias_output

# 4. 对比手动计算结果与系统预测结果
cat("1. 抽取测试集第一个样本:\n", x_input, "\n\n")

```

```

## 1. 抽取测试集第一个样本:
## 0.4810502 0 0 0.4496805 0.2484472 0.5776163 0.5725038 0.07417582

```

```

cat("2. 手动通过公式 ( $w \cdot x + b$ ) 计算的预测值:\n", final_prediction, "\n\n")

```

```

## 2. 手动通过公式 ( $w \cdot x + b$ ) 计算的预测值:
## 0.3812353

```

```

cat("3. neuralnet compute() 函数给出的预测值:\n", nn_pred[1], "\n")

```

```

## 3. neuralnet compute() 函数给出的预测值:
## 0.3812353

```

验证结论: 可以看出, 手动利用提取的参数通过矩阵乘法和激活函数计算得到的结果, 与黑盒函数直接输出的预测值完全吻合。这证明了神经网络寻找参数的本质: 通过反向传播算法不断调整 W 和 b , 直到最终的前向传播计算出的预测误差最小。

问题 3: 课本讨论了 SVM 的分类应用。总结一下 SVM 的回归应用。

课本提供的 `letterdata.csv` 数据集展示了支持向量机 (SVM) 在分类 (Classification) 领域的强大能力。SVM 分类的核心思想是寻找一个超平面 (或通过核函数映射到高维空间中的超平面), 使得不同类别数据点之间的“间隔 (Margin)”最大化。

然而，SVM 思想同样可以推广到回归预测（Regression），称为 支持向量回归（Support Vector Regression, SVR）。

SVR 的理论总结：

1. **ϵ -不敏感损失 (ϵ -insensitive loss)**：传统的线性回归试图让所有点完全落在直线上（最小化残差平方和）。而 SVR 构建了一个以预测函数为中心，宽度为 2ϵ 的“间隔带”（Tube）。只要真实值落在这个带子内，SVR 就认为预测是完全准确的（损失为 0）。只有当数据点超出这个范围时，才计算惩罚。这使得 SVR 对少量的异常值（Outliers）极具鲁棒性。
2. **最大化平坦度**：与分类中最大化 Margin 类似，SVR 的优化目标是寻找一个尽可能“平坦”的回归超平面（即最小化权重向量的范数 $\|w\|^2$ ），从而避免过拟合。
3. **核技巧 (Kernel Trick)**：对于非线性回归问题（如混凝土强度），SVR 同样可以使用核函数（如 RBF 径向基函数），隐式地高维空间中进行线性拟合。

SVR：使用混凝土回归数据

下面我们在 concrete 数据集上训练一个 SVR，看看它的表现。

```
# 使用 e1071 包中的 svm() 进行 SVR 回归预测
# 默认采用径向基核函数 (RBF)，适用于非线性映射
svr_model <- svm(strength ~ ., data = concrete_train)

# 预测
svr_pred <- predict(svr_model, concrete_test)

# 评估 SVR 性能
cat("---- SVR 模型性能 ----\n")
```

```
## --- SVR 模型性能 ---
```

```
cat(" 相关系数 (Cor):", cor(svr_pred, actual_strength), "\n")
```

```
## 相关系数 (Cor): 0.9146048
```

```
cat(" 均方误差 (MSE):", mean((svr_pred - actual_strength)^2), "\n")
```

```
## 均方误差 (MSE): 0.007412789
```

从结果来看，引入核函数的 SVR 模型在当前数据集上的表现甚至可能优于我们刚才构建的简单单节点神经网络，进一步证明了 SVM 及其核方法不仅是强大的分类器，同样也是顶尖的回归工具。“